Learning with AI — A Practice Run

Listen

CSC 240 · C++ DATA STRUCTURES · AI LEARNING TRANSCRIPTS

Learning with AI — A Practice Run

In a data structures course, AI's most common mistake is handing you a finished structure you were supposed to build. This page walks you through a practice conversation so you know how to make AI teach you instead, before your next AI Learning Transcript.

About 30 minutes | Not graded | Six moves to practice | Prepares you for every AI Learning Transcript

ON THIS PAGE

1. Why this page exists
2. Set up before you start
3. Six moves that make AI teach you
4. See it done: a strong conversation
5. Your practice run
6. From practice to your AI Learning Transcript
7. What a full transcript looks like
8. Warning signs
9. Before you submit a transcript

1 Why this page exists

In this course you show what you know **without AI**. Mini Oral Inquiry videos are recorded with no AI open, and in each Major Oral Inquiry you answer questions about your own lab code, trace code you have not seen, and write or defend a short piece of code live. The cumulative final exam is proctored. If AI did your thinking during the week, all three will show it.

> **The CSC 240 trap:** ask an AI for help with a stack, list, tree, or hash table and it will often reach for `std::stack`, `std::list`, `std::map`, or `std::unordered_map`. Those are excellent tools, and they are exactly what the labs ask you to build

> These are excellent tools, and they are exactly what the labs ask you to build yourself. Spotting that substitution, and saying so, is one of the most valuable things you can show in a transcript.

Two recent studies explain why this course grades *how* you use AI rather than whether you do:

- In a field experiment with nearly a thousand students published in PNAS in 2025, students who practiced with an ordinary chatbot did better on practice problems, but once the AI was taken away they scored about 17% worse than students who never had it. A tutor version that gave hints instead of answers largely avoided that drop.
- In a University of Maryland trial of a course AI tutor, most requests asked for information or solutions, and fewer than 1 in 100 asked the tutor for feedback on the student's own attempt. Students offered the tutor finished with lower grades.

The pattern is the same both times: **AI that does the work for you feels helpful and leaves you weaker; AI that makes you do the thinking leaves you stronger.** This page practices the second kind.

2 Set up before you start 5 minutes

Turn on your tool's learning mode (recommended)

The major AI tools now have a mode built for studying that asks you questions and holds back finished answers: **Claude** calls it Learning mode, **ChatGPT** calls it Study mode, and **Gemini** calls it Guided Learning. Where these appear changes as the apps update, so look for them in the tool's mode or style picker. If you can't find one, the **tutor role** prompt in Section 3 does the same job.

Turn off AI autocomplete in VS Code while you work on a lab

If you have installed GitHub Copilot or another AI coding extension, its gray “ghost text” finishes your lines before you have thought them through. That is the answer-machine habit in its quietest form. Disable inline suggestions (or the extension) while you write lab code, and turn it back on later if you like. Major Oral Inquiry sessions are held with AI turned off, so practice that way.

Protect your privacy and your classmates'

- Never paste your student ID, passwords, or other personal information into an AI tool.
- Paste **your own** code only. Pasting a classmate's code into an AI tool shares their work without permission.

3 Six moves that make AI teach you

Copy a prompt, then replace the parts in [brackets] with your own work. Each move is something a good tutor would make *you* do. Strong transcripts use most of them.

1 Set the tutor role

Default chat jumps straight to finished answers. Tell it how to help.

> *Act as my tutor for CSC 240 C++ Data Structures. Do not write code for me. Ask me one question at a time until I find the answer myself, and tell me plainly when my reasoning is wrong.*

2 Predict, then run

Decide what your code will do *before* you run it. A wrong prediction is where the learning is.

> *I predicted this would print [3 5 8], but when I compiled and ran it, it printed [5 8]. Don't fix it. Ask me questions that help me find why my prediction was wrong.*

3 Explain it back

Put the idea in your own words and let the AI find the gap.

> *Here is how I think [how pushBack walks a linked list] works: [your explanation]. What did I get wrong or leave out? Point to the gap; don't give me the full explanation.*

4 Challenge and verify

AI is often confident and wrong. Make it commit to something you can test, then test it.

> *Give me one input where my [removeItem] gives the wrong answer, and tell me what output you expect. Don't fix it. I'll run it and report back.*

5 Get feedback on your own work

Ask for critique, not a rewrite. One bug and one missing test is plenty.

> *Here is my function: [paste your code]. Don't rewrite it. Tell me one possible bug, one test case I'm missing, and one thing that would make it easier to read.*

6 Practice without help

Ask for practice, answer it yourself, and only then check.

> *Give me three short tracing questions on [linked-list pointer updates], one at a time, without the answers. After I answer each one, tell me whether I'm right and why.*

The fastest way to check the AI is your compiler. Asking “are you sure?” often gets the same mistake repeated with more confidence. Compile it, run it, and paste the real output back.

Two extra moves for data structures

+ Draw it first

Sketch the boxes and arrows on paper before you ask. Then describe your drawing and ask the AI to check it.

> *Here is my drawing of the list before and after [operation]: [describe each node and where each pointer points]. Which pointer did I get wrong? Don't draw it for me.*

+ Ask about cost

Every structure in this course trades one cost for another. Make the AI reason with you, not for you.

> *I think [operation] on my [structure] is O([your guess]) because [your reasoning]. Is my reasoning right? If not, give me an input that shows why, and I'll trace it myself.*

Add the course rules to your tutor prompt. Something like: “This is a data structures course. I must implement structures myself with raw pointers and arrays, so don't suggest STL containers. Standard: C++17.” It cuts down the substitution trap.

4 See it done: a strong conversation

A student is practicing Unit 2 ideas with a small linked list (not a lab). The list should hold 3, 5, 8, but it prints `5 8`.

```
void pushBack(int v) {
    Node* n = new Node{v, nullptr};
    if (head == nullptr) { head = n; return; }
    while (head->next != nullptr)
        head = head->next;
    head->next = n;
}
```

STUDENT Move 1 · tutor role

Act as my data structures tutor. I must build this list myself with raw pointers, so don't suggest STL containers and don't write code for me. Ask me questions.

AI

Understood. What did you expect the list to print after pushing 3, 5, and 8, and what did it print?

STUDENT Move 2 · predict, then run

I expected `3 5 8`. It printed `5 8`.

AI

So the first node is missing. Trace the third call, `pushBack(8)`. Before the loop, which node does `head` point to? After the loop?

STUDENT Move 3 · explain back

Before, `head` points to 3. The loop moves `head` itself to 5, so after the call the list starts at 5. Nothing points to the 3 node anymore, so it's lost and never deleted, which is a leak. I should walk a separate pointer and leave `head` alone. Is that it?

AI

Exactly. You could also switch to `std::list`, which handles this for you.

STUDENT Move 4 · pushback

I can't: building the list is the assignment. I used a local `Node* cur = head;` for the walk instead. Now it prints `3 5 8`. Don't rewrite anything, but which cases should I still test?

AI

What happens when you print an empty list, and when you push onto a list with exactly one node?

STUDENT Reflection

Empty prints a blank line and one node prints `3`, both correct when I ran them. I learned that `head` is the list's only anchor, so moving it loses everything before it. **One question I still have:** walking to the end makes `pushBack` O(n). Would keeping a tail pointer make it O(1), and what would I have to update on removal?

The student found the bug by tracing, named the leak, refused the STL shortcut, tested the edge cases, and ended on a cost question. The AI only asked questions. Every output above was checked by actually compiling and running the code.

5 Your practice run about 20 minutes

Activity A · Catch the AI

A student inserted the keys 10, 20, 30, 40, 50, in that order, into an ordinary (unbalanced) binary search tree, then searched for 50. An AI tool explained the cost. One statement is wrong. Find it, and decide how you would prove it.

AI TOOL

(1) Search in a binary search tree is O(log n). (2) With 5 keys, $\log_2 5$ is about 2.3, so finding 50 takes about 3 comparisons. (3) That is why BSTs beat linked lists for searching.

▶ **Show the answer**

Activity B · What kind of request is it?

Label each prompt: *asks for the answer*, *asks for an explanation*, *asks for practice*, *feedback on my work*, or *challenges or verifies*.

1. “Implement a queue class with enqueue and dequeue.”
2. “Why does a circular array queue need to tell full apart from empty?”
3. “Give me three insert sequences and let me draw the resulting BSTs. No answers yet.”
4. “Here's my copy constructor. Don't rewrite it; tell me one case where it could double-free.”
5. “You said my hash table insert is O(1), but my count of probes grows as it fills. Why?”
6. “Convert my code to use `std::vector` so it works.”

▶ **Show the answers**

Activity C · Your own practice conversation

Open your AI tool and hold a short conversation about the structure you are building right now. This one is **not submitted**; it is a rehearsal.

- [ ] Set the tutor role with the no-STL rule, or turn on your tool's learning mode.
- [ ] Draw one operation on paper, describe the drawing, and ask the AI to find the wrong pointer.
- [ ] Predict the output of your driver, run it, and report the result.
- [ ] State the Big-O of one operation with your reasoning, and ask for an input that would prove you wrong.
- [ ] Paste one of your own functions and ask for one bug and one missing test, with no rewrite.
- [ ] End with “One question I still have:”

6 From practice to your AI Learning Transcript

An AI Learning Transcript is this same kind of conversation, started from a lab's AI

Learning Query and submitted as a file. You submit **one per unit**, 10 points each.

Which seed prompt to use

Every lab with a Creative Final Feature has an **AI Learning Query**, and you should do all of them, because your feature grows out of them. Only **one lab's query per unit** is the one you submit as that unit's transcript. The table on the **AI Learning Transcript Overview** page in the **AI Learning Transcripts** module tells you which lab it is for your section. Paste that query as your **first message, exactly as written**. If you want the tutor role too, turn on learning mode or put the tutor text in your tool's standing instructions (often called custom instructions or personal preferences), so the query stays first and unchanged.

What counts as an exchange

One message you write plus the AI's reply. The seed prompt and the AI's first answer to it **don't count**. You need at least **five** follow-up exchanges after that.

Required transcript elements and the moves that satisfy them

Every transcript must include	Moves that get you there
The seed prompt, used as written, as your opening message	—
At least five follow-up exchanges	Any; aim for a mix
At least one exchange where you paste your own code and ask about it	Move 5, Move 2, Draw it first
At least one exchange on something you found difficult, plus a 2–3 sentence summary of what made it click	Move 3, Ask about cost
At least one pushback, where the answer was wrong, incomplete, or didn't match what your compiler did, and why	Move 4, Move 2
A closing reflection: what you learned, where the tool helped or misled you, and one question you still have	Reflection

Submitting: save the conversation as a PDF or Word file named `Unit<N>_AITranscript_<LastName>.pdf`, and upload it to that unit's **AI Learning Transcript** folder, which is separate from the lab folder. Copy the conversation as text (or

use the tool's export); screenshots are hard to read. Include the confused parts too; they are the interesting part.

The Group Lab has its own Group AI Learning Transcript; its instructions are in the **Group Lab** module.

> A transcript in which the AI wrote your structure, and you submitted that code, earns no credit for the work it replaced, and leaves you unable to defend it at a Major Oral Inquiry.

7 What a full transcript looks like

This is the shape of a strong transcript. It uses a made-up practice problem, not one of your labs, and the seed prompt is shortened here; yours is the full AI Learning Query from the designated lab. In your real file, paste every AI reply in full.

```
SEED PROMPT (pasted exactly from the lab; not an exchange)
I am a student in C++ Data Structures studying array-based lists ... Do
not write the full
implementation — walk me through the logic. After you explain, I will
ask follow-up questions.

AI  [first answer, pasted in full]

EXCHANGE 1  When my array fills, I grow it by one slot each time. Here
is my
insert [pasted code]. Without rewriting it, what does each insert cost
as n grows?
AI  [reply]

EXCHANGE 2  I timed 10,000 vs 20,000 inserts with my driver and the
time went
up about 4x, not 2x [pasted output]. Is that the copying you described?
AI  [reply]
MY SUMMARY  Growing by one copies the whole array on every insert, so
the
total work is 1 + 2 + ... + n, which is quadratic ... [2-3 sentences in
my own words]
```

EXCHANGE 3 You suggested switching to std::vector. I can't; this lab asks me
to manage the array myself. Explain why doubling makes the average cost O(1)
so I can implement it.
AI [reply]

EXCHANGE 4 Here is my doubling version [pasted]. Don't fix it: where could my
copy loop leak or double-free?
AI [reply]

EXCHANGE 5 You said deleting the old array before copying is fine. I tried it
and got garbage values [pasted output]. I think the order is wrong. Am I right?
AI [reply]

CLOSING REFLECTION
I learned why doubling gives amortized O(1) inserts and growing by one gives O(n).
The AI helped when it made me add up the copies. It misled me twice: it reached
for std::vector, and it said I could delete the old array before copying.
One question I still have: should the array ever shrink when items are removed,
and when would that cost more than it saves?

Warning signs you are using AI the wrong way

Reflect in ePortfolio Download Print

Open with docReader

Activity Details

You have viewed this topic

A 30-minute, ungraded walkthrough of how to make AI teach you data structures instead of building them for you, including how to spot an AI swapping in STL containers. Moves with copyable prompts, a worked example, a short practice activity, and exactly what every AI Learning Transcript must include. Do it before your next transcript.

Last Visited Oct 16, 2026 11:51 PM